

1 Biểu diễn video và cái giá của thời gian (Video Representation and the Cost of Time)

PHỤ THUỘC C/13/01-dong-cnn (conv, số tham số, FLOPs); C/13/02-tu-chuoi-toi-attention (chi phí bậc hai của attention); B/11-gioi-han-tinh-toan (bảng thông bộ nhớ, ngân sách tính toán).

TÓM TẮT

Mục này trả lời câu hỏi “thêm chiều thời gian vào ảnh thì phải trả giá gì”. Câu trả lời ngắn: bộ nhớ, và nó bị chia cho ba chiều cùng lúc — độ dài clip, độ phân giải, và **batch** — nên tăng một chiều buộc phải giảm chiều khác. Bốn họ kiến trúc (gộp theo khung, hai luồng, tích chập 3D và bản tách $(2+1)D$, video transformer) là bốn cách khác nhau để mua thông tin thời gian với ít bộ nhớ nhất có thể. Đọc xong bạn tính được ngân sách bộ nhớ của một cấu hình video *trước khi* chạy, và biết chiều nào nên hy sinh cho bài toán của mình. Nếu chỉ nhớ một điều: trước khi tin bất kỳ mô hình video nào, hãy chạy baseline “mỗi khung một lần rồi gộp”. Trên rất nhiều tập dữ liệu, baseline đó gần bằng mô hình chuyển động. Chênh lệch còn lại mới là thứ đáng trả tiền.

KIẾN THỨC CẦN CÓ TRƯỚC

- **clip** — một đoạn video ngắn gồm T khung hình liên tiếp (hoặc lấy mẫu thưa), là đơn vị mà mô hình video nhận vào — tương đương “một ảnh” của mô hình ảnh.
- **kích hoạt (activation)** — tensor giá trị trung gian mà mỗi tầng mạng sinh ra — khác với *hàm kích hoạt*, là phép phi tuyến áp lên nó. Khi huấn luyện phải giữ các tensor này trong bộ nhớ để tính gradient ở **lượt ngược** (*backward pass*), nên chúng — chứ không phải tham số — thường là thứ làm tràn bộ nhớ.
- **batch size** — số mẫu xử lý cùng lúc trong một bước cập nhật. Batch lớn cho ước lượng gradient ổn định hơn nhưng tốn bộ nhớ tỉ lệ thuận.
- **batch normalization** — phép chuẩn hoá kích hoạt bằng trung bình và phương sai *tính trong batch*. Batch quá nhỏ thì hai thống kê này rất nhiễu, nên mô hình video hay phải đổi sang chuẩn hoá theo nhóm hoặc theo tầng.
- **FLOPs** — số phép tính dấu phẩy động cần cho một lần chạy; dùng để so chi phí tính toán độc lập với phần cứng. Nó không tỉ lệ chặt với thời gian chạy thật, vì bộ nhớ mới hay là nút thắt.
- **optical flow** — trường vector cho biết mỗi pixel dịch đi đâu giữa hai khung hình liên tiếp — biểu diễn tường minh của chuyển động, dùng làm đầu vào cho nhánh chuyển động của kiến trúc hai luồng.
- **token và chi phí bậc hai** — Transformer chia đầu vào thành n token rồi tính quan hệ giữa mọi cặp, nên chi phí tỉ lệ n^2 . Với video, n nhân thêm số khung hình, và n^2 nhân thêm bình phương số đó.
- **inductive bias** — giả định cài sẵn trong kiến trúc, ví dụ tích chập giả định tính cục bộ và **đẳng biến tịnh tiến** (*translation equivariance*). Bỏ bớt giả định thì mô hình linh hoạt hơn nhưng cần nhiều dữ liệu hơn để tự học lại điều đã bị bỏ.

Đặt vấn đề

Cách hiển nhiên nhất để xử lý video là coi nó như một khối ba chiều và dùng **tích chập ba chiều** (*3D convolution*) thay cho tích chập hai chiều. Cách này đúng về mặt hình thức và sai về mặt kinh tế: số phần tử phải giữ trong bộ nhớ nhân với số khung hình, số tham số của mỗi kernel nhân với chiều dài thời gian, và ngân sách vốn đã chật của một mô hình ảnh lập tức vỡ. Vì thế toàn bộ lĩnh vực này không phải là câu chuyện “kiến trúc nào tốt hơn” mà là câu chuyện “mua thông tin thời gian bằng cách nào cho rẻ”.

Nhưng còn một lý do sâu hơn khiến ta không nên coi thời gian là trục không gian thứ ba. Trục thời gian có *hướng*: đảo ngược nó thì “mở cửa” thành “đóng cửa”, trong khi lật ngược trục x của một ảnh vẫn cho một ảnh hợp lệ của cùng một cảnh. Trục thời gian cũng có *đơn vị khác*: 16 khung

hình ở 30 Hz là nửa giây, còn 16 pixel là một chi tiết nhỏ — không có lý do gì để cùng một kernel $3 \times 3 \times 3$ là kích thước hợp lý ở cả ba trục. Hai bất đối xứng này là nguyên nhân sâu xa của mọi thiết kế tách trục mà ta sẽ gặp.

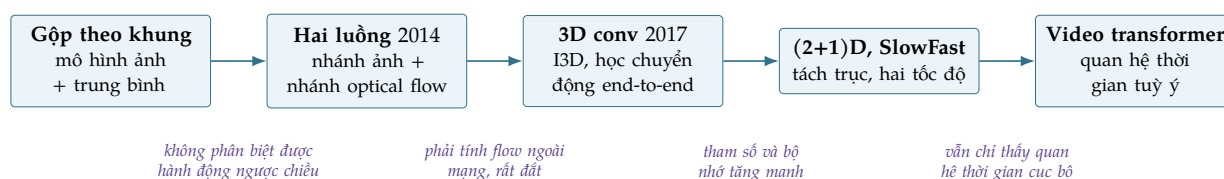
Học xong mục này bạn làm được gì mà trước đó không làm được? Cầm một yêu cầu như “phân loại hành động trên clip 4 giây, chạy được trên GPU 8 GB”, bạn suy ra ngay cấu hình khả thi và thứ phải hy sinh. Không phải thử ngẫu nhiên tới khi hết bộ nhớ.

TRỰC GIÁC

Ngân sách bộ nhớ của video giống một tấm chăn ngắn kéo theo ba hướng: độ dài clip, độ phân giải, và batch size. Tích của ba đại lượng đó gần như là một hằng số do phần cứng ấn định, nên mọi quyết định kiến trúc thực chất là quyết định kéo chăn về phía nào. Điều đáng nói là ba hướng ấy không tương đương về mặt khoa học: giảm batch làm hỏng thống kê của **chuẩn hoá theo batch** (*batch normalization*), giảm độ phân giải làm mất vật nhỏ, còn giảm độ dài clip làm mất chính thứ ta đang muốn học. Không có hướng nào miễn phí, và biết trước điều đó tiết kiệm hàng tuần thí nghiệm.

↔ **ràng buộc bởi** $B/11$ — tích độ dài clip $\times$ độ phân giải² $\times$ batch bị chặn bởi bộ nhớ thiết bị — cùng một trần đã chi phối mọi lựa chọn ở chương giới hạn tính toán.

A. Bốn họ biểu diễn, và nút thắt mà mỗi họ gỡ



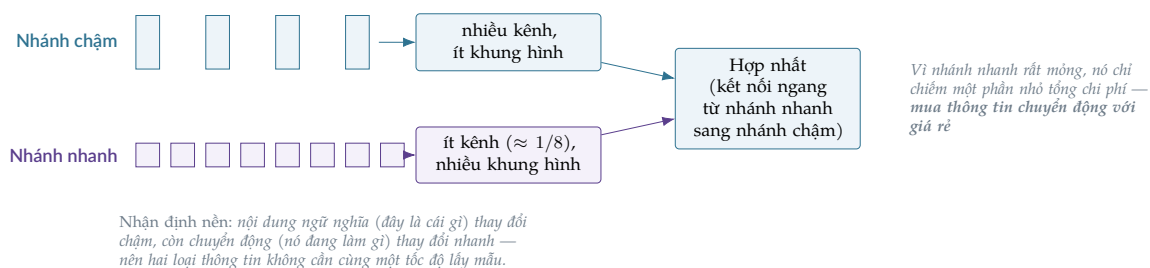
Hình 1. Dòng thời gian biểu diễn video, đọc theo nút thắt được gỡ. Nhấn tím dưới mỗi mũi tên là *vấn đề của bước trước* — đó là thứ khiến bước sau ra đời. Chú ý rằng đây không phải một thang chất lượng: bước đầu tiên vẫn là baseline mạnh và bắt buộc phải chạy.

Hình 1 cho thấy mạch phát triển, còn Bảng 1 cho thấy cái mà mạch đó che đi: mỗi họ giả định một điều khác nhau về bản chất của chuyển động, và chính giả định — chứ không phải số tầng — quyết định nó hỏng ở đâu.

Bảng 1. Năm cách biểu diễn video. Cột chi phí ghi theo bậc độ lớn tương đối so với một mô hình ảnh chạy trên một khung hình, không phải số đo thực nghiệm.

Cách	Giả định	Chi phí	Được	Hỏng khi nào
Mỗi khung + gộp	Thời gian không quan trọng lắm	$\sim T \times$	Rẻ, tái dùng nguyên mô hình ảnh	Không phân biệt được hành động ngược chiều (mở/đóng, nhặt/đặt)
Hai luồng	Chuyển động tách được thành một nhánh riêng [1]	$\sim 2T \times + \text{flow}$	Rõ ràng, hiệu quả với ít dữ liệu	Phải tính optical flow ngoài mạng; flow sai thì nhánh đó thành nhiễu
3D conv	Không-thời gian đối xứng [2]	$\sim 3T \times$	Học chuyển động đầu-cuối	Tham số và bộ nhớ tăng mạnh; giả định đối xứng vốn sai
(2+1)D	Trục thời gian tách được khỏi trục không gian [3]	$\sim 1,5T \times$	Cùng số tham số nhưng thêm một hàm kích hoạt, dễ tối ưu hơn	Chuyển động thực sự xoắn không-thời gian (xoáy, biến dạng nhanh)
Video transformer	Quan hệ thời gian tùy ý, không cần cục bộ	bậc hai theo số token	Mạnh nhất khi đủ dữ liệu	Chi phí bậc hai theo T ; rất đói dữ liệu

Một họ nữa nằm ngoài bảng vì nó là một ý tưởng chéo chứ không phải một loại tích chập: **Slow-Fast** chạy *hai* nhánh trên cùng video — một nhánh tốc độ khung thấp nhưng nhiều kênh để nắm ngữ nghĩa, một nhánh tốc độ khung cao nhưng ít kênh để nắm chuyển động [4]. Ý nghĩa của thiết kế này: *nội dung ngữ nghĩa thay đổi chậm, còn chuyển động thay đổi nhanh, nên hai loại thông tin không cần cùng một tốc độ lấy mẫu*. Nhánh nhanh rất mỏng: bài báo dùng khoảng một phần tám số kênh. Nó vì thế chỉ chiếm một phần nhỏ tổng chi phí. Đây là ví dụ đẹp của việc dùng cấu trúc bài toán để mua thông tin rẻ hơn.



Hình 2. SlowFast dùng cấu trúc của tín hiệu để mua thông tin rẻ hơn. Thay vì tăng đồng loạt tần số lấy mẫu cho toàn mạng — cách tốn kém nhất — nó tách thành hai nhánh có tần số và bề dày kênh ngược nhau, rồi hợp nhất. Nếu nhận định nền sai với dữ liệu của bạn (ngữ nghĩa thay đổi nhanh, chẳng hạn cảnh cắt dựng liên tục) thì lợi thế chi phí biến mất.

Hình 2 đáng đặt cạnh Hình 3: cả hai đều là ví dụ của cùng một chiến lược — đọc ra một bất đối xứng thật của bài toán rồi đưa nó vào kiến trúc, thay vì trả tiền đồng đều cho mọi chiều.

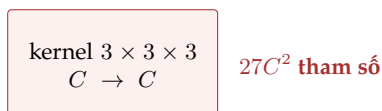
B. Ba phép tính quyết định mọi thứ

- **Bộ nhớ của một tensor kích hoạt.** Một clip 16 khung, 224×224 , 64 kênh, fp32: $16 \times 224 \times 224 \times 64 \times 4 \text{ byte} \approx 205 \text{ MB}$ — cho *một* tensor, ở *batch* bằng 1. Lướt ngược phải giữ kích hoạt

của mọi tầng, nên con số này nhân với số tầng nông. Đây là lý do thực tế vì sao mô hình video hầu như luôn chạy ở batch nhỏ, và vì sao chuẩn hoá theo batch trở thành vấn đề (xem C/12-co-che-huan-luyen/02-lam-cho-training-chay-duoc).

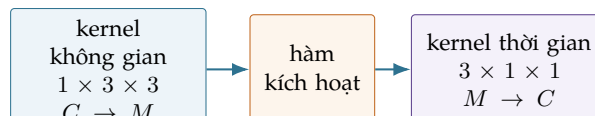
- **Vì sao (2+1)D không tốn thêm tham số.** Một kernel 3D $3 \times 3 \times 3$ từ C kênh sang C kênh có $27C^2$ tham số. Bản tách gồm hai kernel. Kernel không gian $1 \times 3 \times 3$ đi từ C sang M kênh, tốn $9CM$; kernel thời gian $3 \times 1 \times 1$ đi từ M về C , tốn $3MC$. Tổng là $12CM$. Cho hai vế bằng nhau: $M = 2,25C$. *Ý nghĩa:* với cùng ngân sách tham số, ta mua được một hàm kích hoạt bổ sung ở giữa và một bài toán tối ưu dễ hơn — lợi ích đến từ dạng của hàm, không từ sức chứa (capacity) [3].

Tích chập 3D



Một khối duy nhất trộn cả ba trục, ngâm giữ định thời gian đối xứng với không gian

Bản tách (2+1)D



tổng $12CM$; cho bằng $27C^2$ thì $M = 2,25C$

Cùng ngân sách tham số, nhưng mua thêm được một hàm kích hoạt ở giữa — lợi ích đến từ dạng của hàm, không từ sức chứa

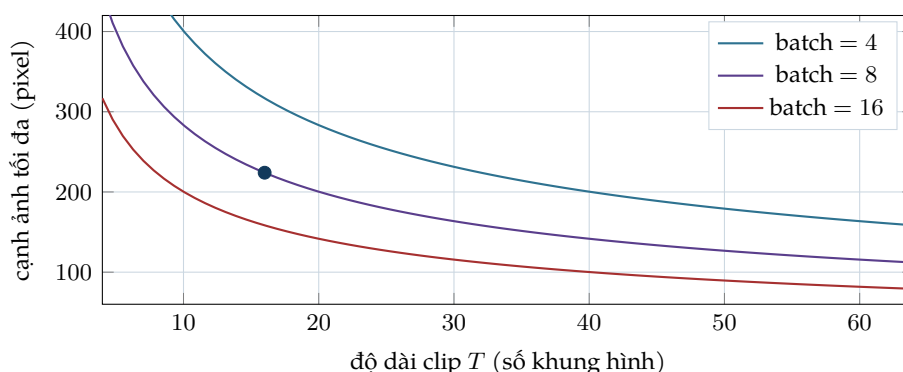
Hình 3. Phân rã (2+1)D vẽ theo khối. Kernel 3D duy nhất được tách thành một kernel chỉ tác động theo không gian rồi một kernel chỉ tác động theo thời gian, với số kênh trung gian M chọn sao cho tổng tham số bằng bản gốc. Phép tách phản ánh đúng bất đối xứng của bài toán: trục thời gian có hướng và có đơn vị khác, nên không có lý do để cùng một kernel đối xứng là hợp lý ở cả ba trục.

Hình 3 làm rõ vì sao đây không phải mẹo tiết kiệm: nếu chỉ để tiết kiệm thì người ta đã giảm số kênh, chứ không giữ nguyên tham số rồi chèn thêm một phi tuyến.

- **Vì sao attention trên video đắt tới mức đó.** Một ViT chia ảnh 224×224 thành $14 \times 14 = 196$ token, nên ma trận attention có $196^2 \approx 3,8 \times 10^4$ phần tử. Với clip 16 khung, số token thành 3136 và ma trận thành $9,8 \times 10^6$ — gấp 256 lần, đúng bằng 16^2 .

↪ **ràng buộc bởi** C/13/03 — video transformer bỏ thêm một tầng thiên lệch quy nạp nữa — lần này theo trục thời gian — nên nó đòi dữ liệu hơn cả ViT trên ảnh.

Ý nghĩa: chi phí bậc hai theo số token nghĩa là bậc hai theo độ dài clip, nên mọi video transformer thực dụng đều phải phân tách attention (theo thời gian rồi theo không gian) hoặc giới hạn cửa sổ.



Hình 4. Đường đẳng ngân sách bộ nhớ. Vì lượng kích hoạt tỉ lệ với $T \cdot R^2 \cdot B$, một ngân sách cố định cho $R \propto 1/\sqrt{TB}$. Điểm xanh đậm là cấu hình tham chiếu (16 khung, 224 px, batch 8). Hình cho thấy điều mà một bảng số không cho thấy: muốn gấp đôi độ dài clip, bạn chỉ phải giảm cạnh ảnh khoảng 30%, nhưng muốn gấp bốn thì phải giảm một nửa — và đường cong dốc rất nhanh ở phía trái.

Hình 4 nên dùng như một công cụ chứ không phải một minh họa: khi ai đó đề nghị “tăng độ dài clip lên 64 khung”, hãy chỉ ngay vào đường cong và hỏi họ định trả bằng độ phân giải hay bằng batch.

C. Giới hạn và trường hợp hỏng

- **Rất nhiều tập dữ liệu không thực sự cần thời gian.** Nhiều tập dữ liệu hành động phổ biến có thể đoán đúng phần lớn chỉ từ *bối cảnh* của một khung hình — thấy bể bơi thì đoán “bơi”. Đây là quan sát được lặp lại rộng rãi trong lĩnh vực, và hệ quả là một mô hình thời gian tinh vi có thể trông tốt vì lý do hoàn toàn khác với lý do bạn nghĩ.
- **Optical flow là một phụ thuộc ngoài mạng.** Nhánh chuyển động của kiến trúc hai luồng đòi hỏi flow được tính trước, thường tốn hơn cả mô hình chính. Chất lượng flow khi đó thành trần cho toàn hệ thống — đúng mẫu “đẩy độ khó sang chỗ khác”.
- **Batch nhỏ phá thống kê chuẩn hoá.**
 ⇔ **hệ quả của** C/12/02 — chuẩn hoá theo batch ước lượng trung bình và phương sai từ chính batch, nên batch nhỏ làm ước lượng nhiễu — đúng cơ chế đã bàn ở phần làm cho huấn luyện chạy được.
 Mô hình video thường chạy batch 2–8 trên mỗi GPU, nên batch norm cho ước lượng phương sai rất nhiễu; đây là lý do các kiến trúc video hay dùng chuẩn hoá theo nhóm hoặc theo tầng [5].
- **Lấy mẫu khung là một siêu tham số ẩn.** Lấy 16 khung liên tiếp, hay 16 khung rải đều trên 4 giây, là hai bài toán khác nhau; và cách lấy mẫu lúc đánh giá thường khác lúc huấn luyện, nên số công bố phụ thuộc vào một chi tiết ít khi được nêu rõ.

CẠM BẮY

Trước khi so sánh bất kỳ hai kiến trúc video nào, hãy chạy baseline rẻ nhất: lấy một khung hình ngẫu nhiên, chạy mô hình ảnh, rồi trung bình dự đoán trên vài khung. Nếu baseline này đạt gần bằng mô hình thời gian, thì tập dữ liệu của bạn không đo cái bạn tưởng nó đo, và mọi so sánh giữa các mô hình thời gian phía trên đều là nhiễu. Đây là bài kiểm tra rẻ nhất và bị bỏ qua nhiều nhất trong cả lĩnh vực.

CẦN NHỚ

Ngân sách bộ nhớ của video chia cho ba chiều nhân với nhau: độ dài clip $\times$ độ phân giải² $\times$ batch. Mọi lựa chọn kiến trúc vì thế là lựa chọn hy sinh chiều nào. Và trước khi trả tiền cho chiều thời gian, hãy chứng minh rằng bài toán của bạn thật sự cần nó, bằng baseline một khung hình. [T2]

D. Kết nối

B/11-gioi-han-tinh-toan là nơi ngân sách bộ nhớ và băng thông được xử lý tổng quát; mục này chỉ là trường hợp riêng nhưng khác nghiệt nhất. C/13-kien-truc-backbone/03-vit-va-inductive-bias giải thích vì sao video transformer đối dữ liệu: bỏ inductive bias theo trục thời gian còn tốn dữ liệu hơn bỏ theo trục không gian. C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc tiếp nối trực tiếp bằng nhánh optical flow — thứ mà kiến trúc hai luồng dựa vào — và bằng ràng buộc nhân quả, thứ loại bỏ phần lớn các kiến trúc trong bảng trên khi bạn phải chạy streaming.

❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao coi thời gian như trục không gian thứ ba là một giả định sai, và sai đó biểu hiện thành lỗi gì trong thực tế?
2. Tính lượng bộ nhớ cho một tensor kích hoạt clip 32 khung, 112×112 , 128 kênh, fp16. Nếu vượt ngân sách, bạn cắt chiều nào trước và vì sao?
3. (2+1)D có cùng số tham số với 3D conv nhưng thường tốt hơn. Lợi ích đó đến từ đâu, nếu không phải từ sức chứa của mô hình?
4. Vì sao chi phí attention trên clip 16 khung gấp 256 lần trên một ảnh, chứ không phải 16 lần?
5. SlowFast dựa trên nhận định nào về bản chất của tín hiệu? Nếu nhận định đó sai với dữ liệu của bạn thì kiến trúc hồng thể nào?
6. Baseline “một khung hình rồi gộp” đạt 92% so với mô hình 3D đạt 94%. Bạn kết luận gì, và bước tiếp theo là gì?

Nguồn

- [1] K. Simonyan and A. Zisserman, “Two-Stream Convolutional Networks for Action Recognition in Videos,” in *NeurIPS*, 2014.
- [2] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” in *CVPR*, 2017.
- [3] D. Tran, H. Wang, L. Torresani, et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” in *CVPR*, 2018.
- [4] C. Feichtenhofer, H. Fan, J. Malik, et al., “SlowFast Networks for Video Recognition,” in *ICCV*, 2019.
- [5] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.